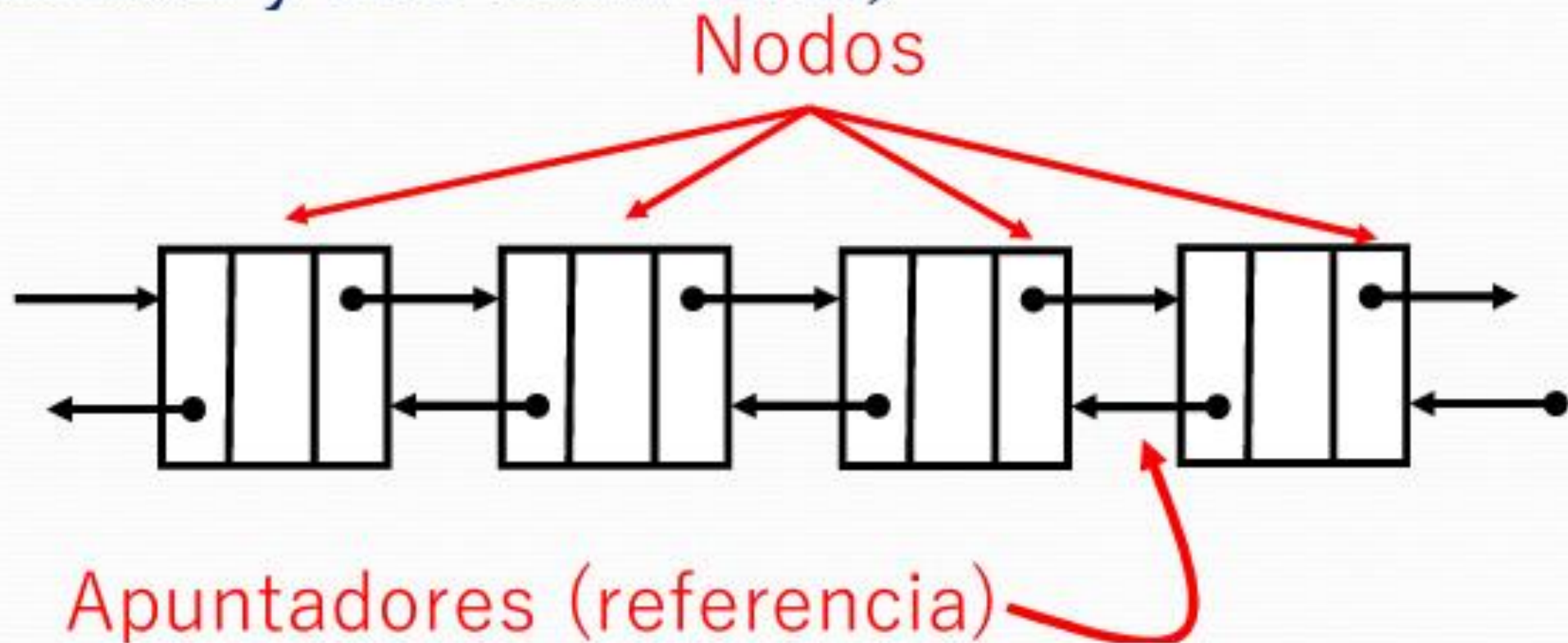
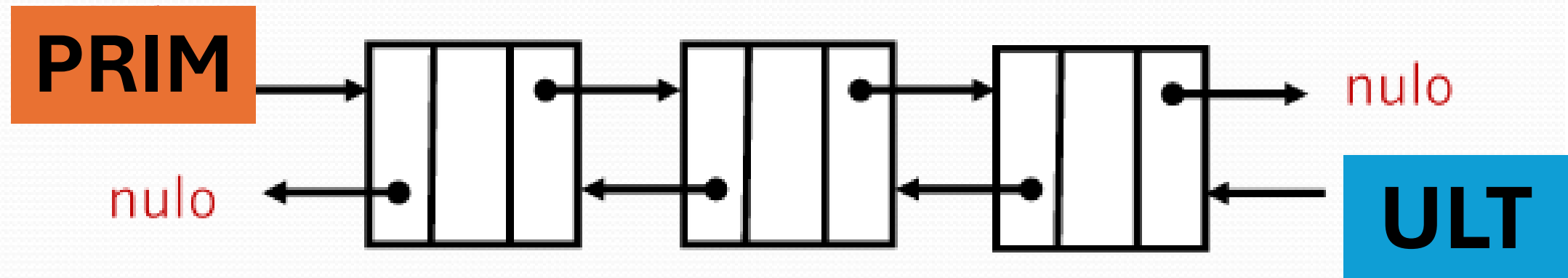


Listas Doblemente encadenadas

Es una estructura de datos dinámica que se compone de un conjunto de nodos en secuencia enlazados mediante dos apuntadores (uno hacia adelante y otro hacia atrás)

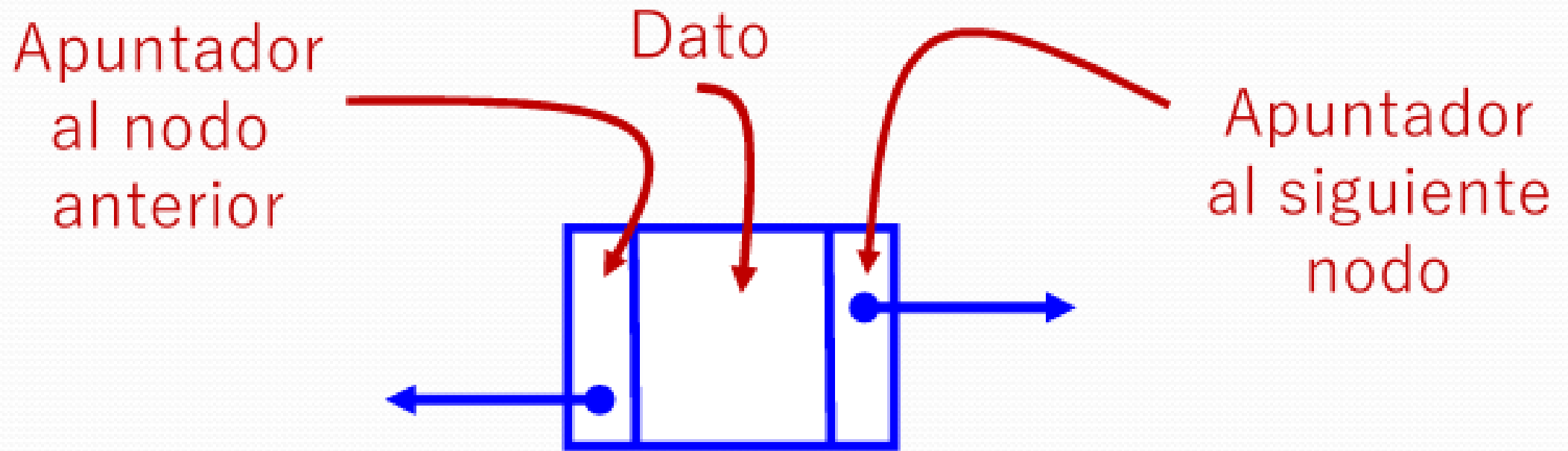


- La *lista doble* tiene un apuntador inicial y un apuntador final
- El primero y el último nodos de la lista apuntan a nulo



Cada nodo tiene 3 secciones:

1. *Dato (puede ser simple o compuesto)*
2. *Apuntador o referencia que enlaza al siguiente nodo en secuencia lógica*
3. *Apuntador que enlaza al nodo anterior*

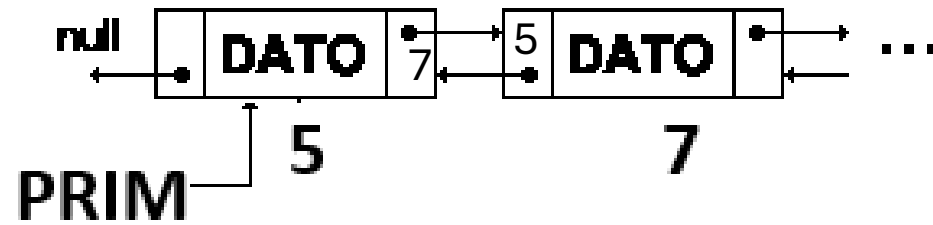


```
// Definición de la estructura del nodo  
struct Nodo {  
    int dato;  
    struct Nodo* siguiente;  
    struct Nodo* anterior;  
};
```

Operaciones básicas con listas doblemente enlazadas

- Añadir o insertar elementos.
- Buscar o localizar elementos.
- Borrar elementos.
- Moverse a través de la lista, siguiente y anterior.

Insertar un nodo al INICIO de la lista



Nuevo (p)

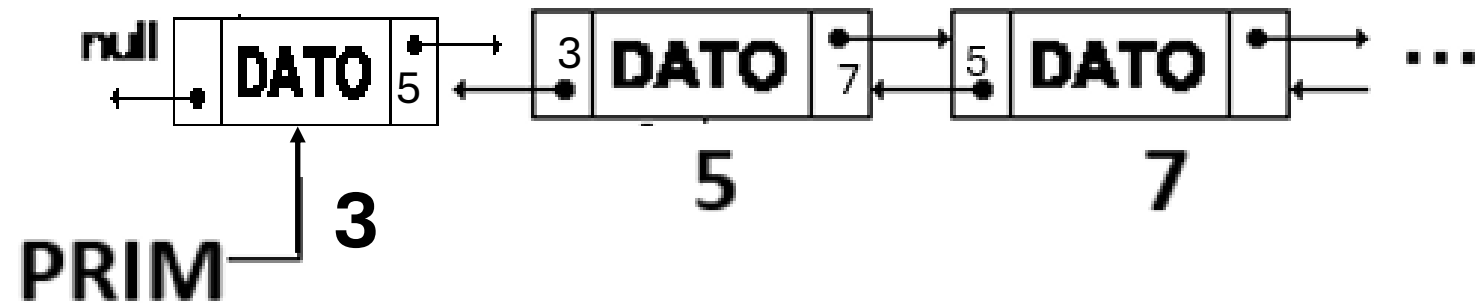
*p.dato= dato

*p.anterior=NULL

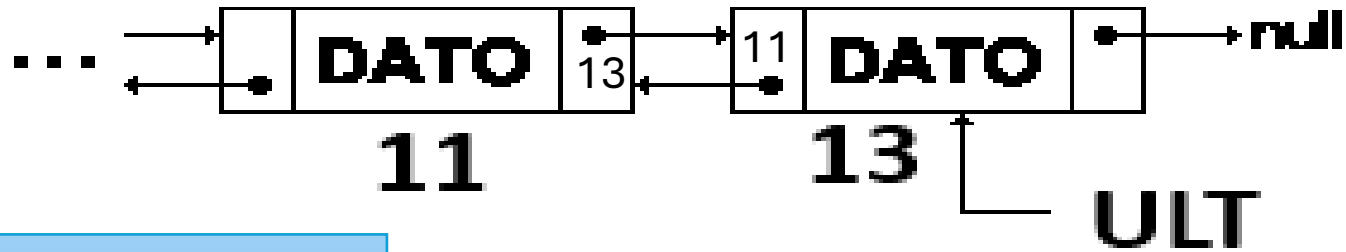
*p.posterior=prim

*prim.anterior=p

prim=p



Insertar un nodo al FINAL de la lista



Nuevo (p)

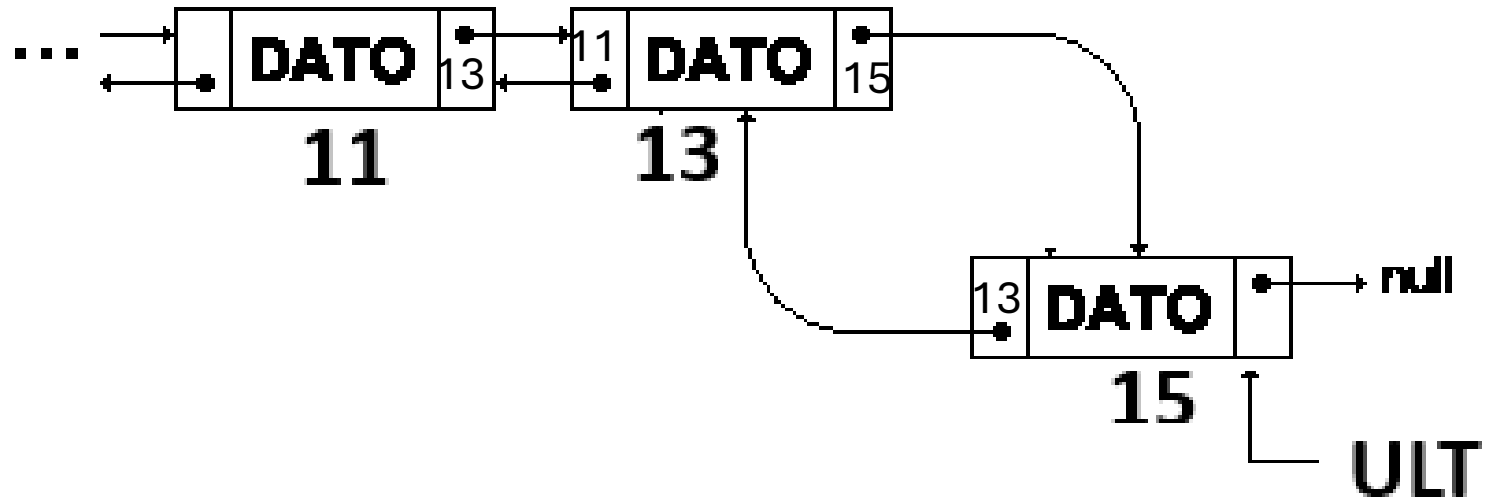
*p.dato= dato

*p.posterior=NULL

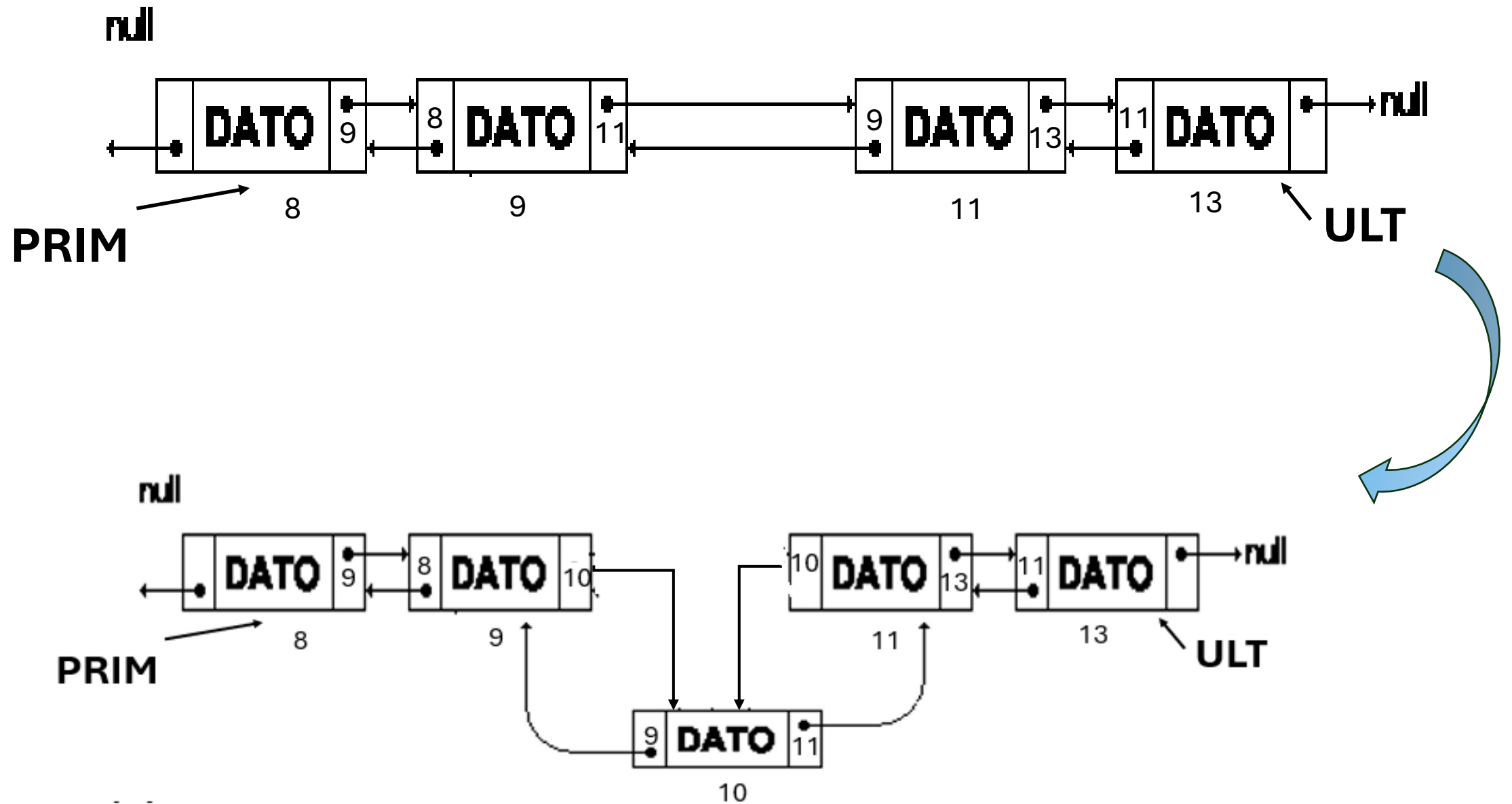
*p.anterior=ult

*ult.posterior=p

ult=p



Insertar un nodo al MEDIO de la lista

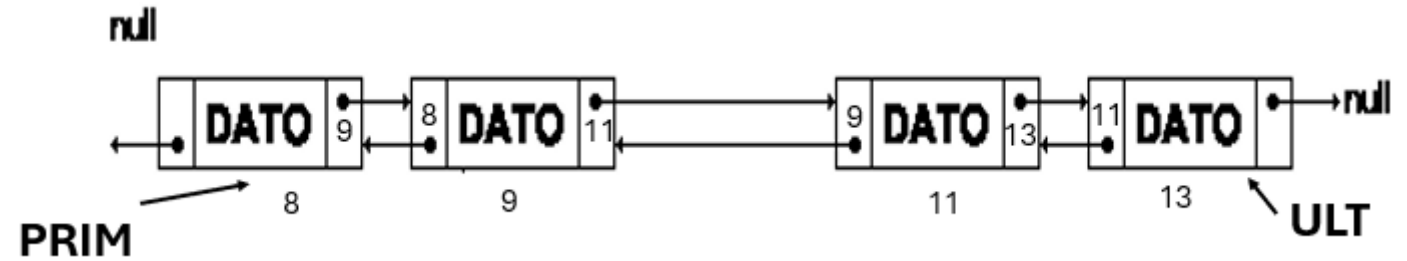


.....

t= prim;

i=1;

```
mientras (t <> null) y (i < pos) hacer
    anterior=t;
    t=*t.posterior;
    i++;
fmientras
```



Si t= null escribir (“No se encontró posición”);

sino

nuevo (p)

si p=null escribir (“No hay espacio”);

sino

Ingresar (dato);

*p.dato=dato;

*p.anterior=anterior;

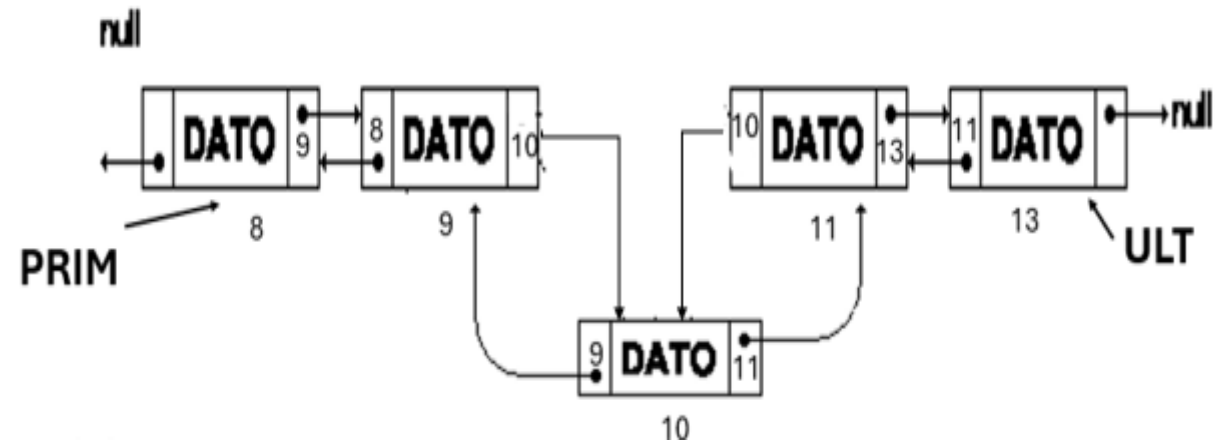
*p.posterior=t;

*t.anterior=p;

*anterior.posterior=p;

fsi

fsi



BUSCAR un elemento en la lista

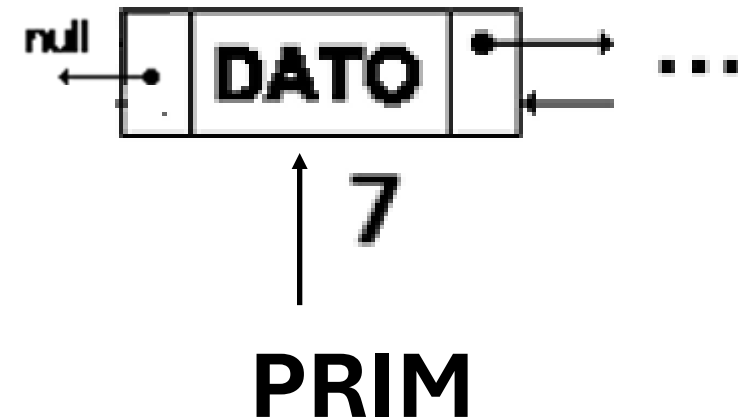
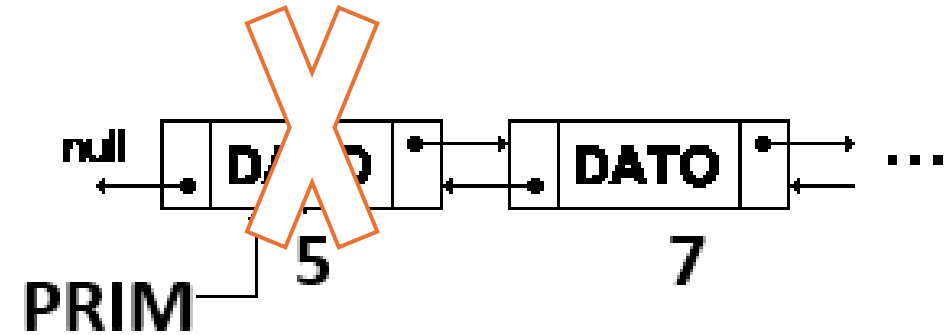
.....

Ingresar (dato-bus);

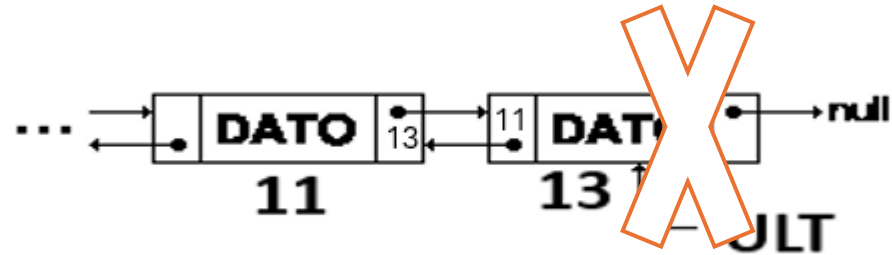
```
si *prim.dato=dato-bus escribir ("Elemento encontrado");
sino
  si * ult.dato=dato-bus escribir ("Elemento encontrado");
  sino
    t=prim;
    mientras (t <> null) y (*t.dato<>dato-bus) hacer
      t=*t.posterior;
    fmientras
      si (t= null) escribir (" Elemento no encontrado")
      sino escribir (" Elemento encontrado")
    fsi
  fsi
fsi
```

Eliminar el primero elemento en la lista

```
p= *prim.posterior  
Liberar(prim)  
*p.anterior=NULL  
prim=p
```



Eliminar el ultimo elemento en la lista



```
p= *ult.anterior  
Liberar(ult)  
*p.posterior=NULL  
ult=p
```



Eliminar un elemento en el medio de la lista

.....

t= prim;

i=1;

mientras (t <> null) y (i < pos) hacer

 anterior:=t;

 t:=*t.posterior;

 i++;

fmientras

Si t= null escribir ("No se encontró posición");

sino

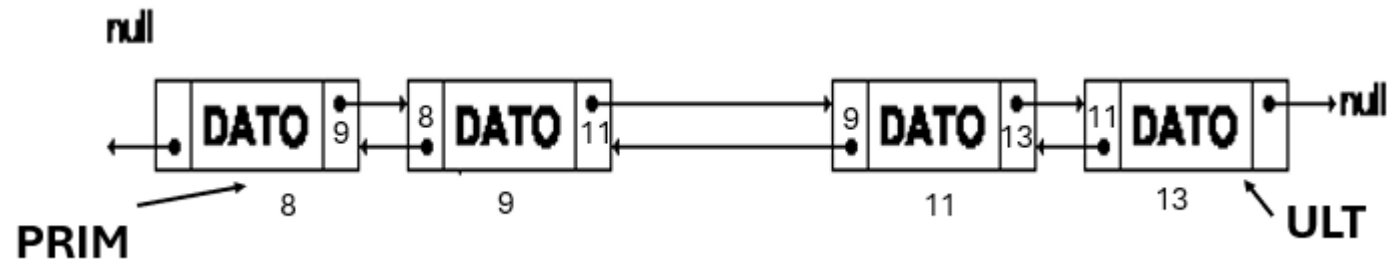
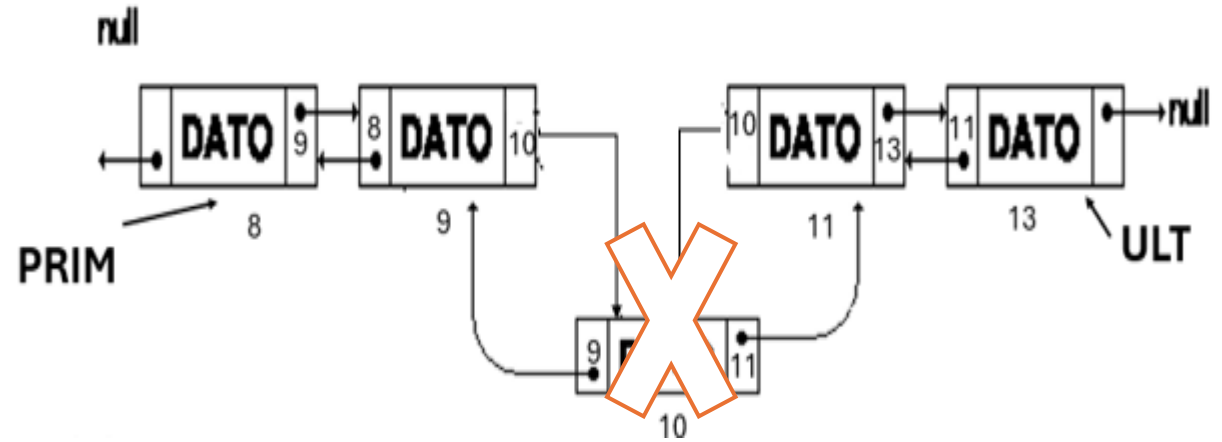
 *anterior.posterior:=*t.posterior;

 aux:=*t.posterior;

 *aux.anterior:=anterior;

 liberar(t);

fsi



Actividad 1:

- Analizar el siguiente código correspondiente a una lista doblemente encadenada.
- Usar comentarios para explicar que hace en cada caso.

Tiempo: 15 minutos



Actividad 2: Responder a las siguientes preguntas

¿Cómo se generan los nodos?

¿Cómo realiza el recorrido en ambos sentidos?

¿Utiliza punteros externos?

Tiempo: 5 minutos



Actividad 3: Modificar el programa considerando:

- Uso de punteros externos en la creación de los elementos de la lista
- Recorrer la lista en ambos sentidos a partir de los punteros externos.

Tiempo: 40 minutos



Listas Circulares

- Una **lista circular** es una estructura de datos lineal donde el último elemento apunta al primer elemento en lugar de terminar en NULL. Esto crea un ciclo o "anillo" continuo.
- Existen principalmente dos tipos: las **simples** (recorrido unidireccional) y las **dobles** (el último apunta al primero y viceversa para moverse en ambas direcciones)